TOOLS.md — Fusion 360 MCP tool reference

Argument safety: all arguments are emitted as proper Python literals, so Windows paths (`C:\\parts\\x.step`), apostrophes, and quotes are handled correctly and cannot alter the code that runs inside Fusion.

Unit rule: every argument is in **millimeters**. Internally tools convert to Fusion's centimeters. When you (or Claude) write raw code, the API is in cm.

Face selection rule: `top/bottom` = max/min Z, `front/back` = min/max Y, `right/left` = max/min X. Chosen by extreme face-centroid position. Reliable on box-like parts; verify with `view_model` on complex geometry.

Core

`ping_fusion()`

Liveness + busy check, served on a **separate channel** from geometry execution — so it answers even while a long op runs on the main thread (a hung op no longer blocks ping). Reports `busy` and `current_job`.

`get_state()`

Cheap, guaranteed-fast doc state from the listener cache: active document name, lineage id, and body/sketch/component/timeline counts. Does **no geometry work** and is served without the main thread — safe to call during a heavy op or after a suspected hang. Use first for crash recovery. (For full per-body detail, `get_model_state`.)

`heartbeat()`

Crash-vs-busy check with explicit recovery instructions. Same liveness data as `ping_fusion`, plus guidance: if it errors, restart the add-in; if `busy=True`, poll `job_status`.

`run_async(code)`

Start a long operation **without waiting** — returns a `job_id` immediately. Poll `job_status(job_id)` for incremental progress and the final result. Use for anything expected to exceed ~90s (large patterns,

heavy booleans, full-assembly rebuilds). Same injected helpers and unit rules. Call `progress(msg)` in your code to mark stages visible while it runs.

`job_status(job_id)`

Structured result of an async job: `status` (queued/running/ok/error/timeout), `elapsed_ms`, stdout collected **so far** (works mid-run), and traceback on failure. Poll until status is ok/error/timeout.

`run_fusion_code(code)`

Execute arbitrary Python in Fusion via the **bounded sync path**: waits up to ~100s and returns a structured envelope (status / elapsed / stdout / traceback). If the job outlives the wait it keeps running and stays pollable — the response tells you the job id. `adsk`, `app`, `ui` in scope, plus injected helpers (below). **Use `mm(x)` for every length** instead of hand-dividing by 10.

The docstring also documents the **sketch-plane axis mapping** (XY/XZ/YZ and offset planes) so the XZ→world-−Z inversion doesn't cost you a debugging cycle.

Injected helpers (available in `run_fusion_code` and `run_fusion_code_safe`, no import needed): - `mm(x)` / `cm(x)` — mm→cm for values you pass to the API. - `to_mm(x)` / `to_mm2(a)` / `to_mm3(v)` — cm→mm for reporting. - `assert_scale(max_mm=2000)` — raise if any body is 10×-too-big (units slip). - `total_volume()` — summed cm³ of all bodies; capture **before** a boolean. - `report_volume_delta(before, label)` — print the change and **scream if ~0** (silent no-op boolean). - `capture_state(target_body)` / `report_volume_delta(before, label, target_body)` — the correct before/after pair. Tracks body count **and** the target body's own volume, so a volume-neutral **Join** is no longer flagged as a false no-op. - `fillet_volume_sanity(r_mm, edge_len_mm, delta_cm3)` — catches a fillet that propagated beyond its edges. - `sliver_face_count(body, min_area_mm2=0.05)` — threshold raised; prefer `smallest_faces(body, n=5)`, which prints the actual smallest areas. - `P_xy(x,y)` / `P_xz(x,z)` / `P_yz(y,z)` — world mm → correct sketch point. - `verify_plane_mapping(sketch)` — print a plane's measured local→world basis. - `retire_body(body)` — the safe replace-a-body idiom. - `edge_ok(e)` / `safe_edges(body)` / `edge_face_pair(e)` — degenerate-edge guards and the surface-type-pair classifier. - `path_report(path)` — segment count + total length, so a mis-chained path is visible (a short sweep is otherwise silent). - `refresh_geometry()` — force a recompute and report whether volumes moved. - `log_sketch_areas(sketch)` — print each profile area in mm² (catches scale slips). - `delete_face(face)` — delete one face the way Fusion accepts (`DeleteFaceFeatures`).

Recommended pattern for any cut/join in raw code:

```
_b = total_volume()
# ... do the boolean ...
report_volume_delta(_b, 'mycut'); assert_scale()
```

Known-unstable (refused by the listener): move-face via `defineAsFreeMove` (hard-crashes the host) and `defineAsOffset` (missing on this build). Use parametric edits / `offsetFeatures` / shell instead.

`run_fusion_code_safe(code)`

Transactional `run_fusion_code`: records the timeline marker and, on error, rolls back and deletes features created after it. Same injected helpers.

`clear_document()`

Delete all bodies, sketches, and timeline features. Returns design to empty.

Parameters

`set_parameter(name, value_mm)`

Create or update a user parameter (mm). Build parametric-first for anything that might change later.

`list_parameters()`

List user parameters and their expressions.

Primitives (root component, XY plane)

> **Auto-observability:** the material-removing tools (`drill_hole`, `hole_on_face`, `tapped_hole`, `shell_body`) automatically report the **volume delta** of the operation and **warn if it's ~zero** (a silent no-op cut that didn't touch the body). `shell_body` also reports sliver-face count. You no longer need to probe volume yourself to confirm a cut worked.

`create_box(length_mm, width_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Rectangle on XY extruded up by height. Returns new body name.

`create_cylinder(diameter_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Circle on XY extruded up.

`drill_hole(diameter_mm, x_mm, y_mm, depth_mm, body_index=0)`

Cut a hole straight down from XY at (x, y) to depth.

`fillet_edges(radius_mm, body_index=0, which="all")`

Round edges. `which`: `all` / `vertical` (vertical-running edges) / `top` (edges lying on the top face plane). Warns if nothing matched.

`shell_body(thickness_mm, body_index=0, open_top=True)`

Hollow to a wall thickness. `open_top=True` removes the top face (open box); `False` makes a closed hollow body.

`tapped_hole(thread_size, x_mm, y_mm, depth_mm, body_index=0, tap_drill_only=True)`

Tapped hole on the top (XY) plane, cutting down. `thread_size` e.g. `M3`, `M3x0.5`, `M4`, `M5`, `M6`, `1/4-20`, `10-32`, `#6-32`. - `tap_drill_only=True` (default): cuts only the correctly-sized tap-drill clearance hole — best for 3D-printed and hand-tapped parts. - `False`: also applies a **modeled** thread (visualization / CNC); best-effort, may be skipped with a warning if Fusion can't match the designation. Known tap-drill sizes include metric M1.6–M12 and common imperial; unknown sizes fall back to M3's drill — pass an explicit size you know is in the table, or ask to extend it.

Face-referenced features

`sketch_on_face(component_name="", which="top", body_index=0)`

Select a face by orientation and stash it. Empty `component_name` = root. Call `hole_on_face` immediately after (single-slot; don't interleave).

`hole_on_face(diameter_mm, u_mm, v_mm, depth_mm, through=False)`

Drill into the stashed face. `(u, v)` are mm offsets from the face centroid along the face's own axes. `through=True` cuts all the way.

Measurement & state

`get_model_state()`

Comprehensive design state in one call. Per body: name, validity, volume (cm³), bbox (mm), face/edge counts, and **sliver-face count** (flags a bad boolean). Plus body/sketch counts and the **timeline feature names**. Replaces hand-rolled introspection; makes crash recovery a one-call operation.

`measure_distance(which_a, which_b, body_index=0)`

True centroid-to-centroid distance (mm) between two oriented faces. Use to verify wall thickness, overall size, gaps — more trustworthy than the bbox.

`cross_section(body_index=0, axis="z", at_mm=None, span_axis="x", samples=41, at_a_mm=None, at_b_mm=None)`

Scan a line of points through a body and report solid/void as an **ASCII row** (`#`=solid, `.`=void) plus the **mm coordinates of every solid/void boundary**. The best non-visual diagnostic for trapped voids, missed cuts, and wall thickness. Holds the two non-span axes at their mid-value (or at `at_mm` along `axis`) and sweeps `span_axis` across the bbox. **Pin the held axes explicitly** with `at_a_mm` / `at_b_mm` to scan through a specific feature — e.g. a vent tube at x=3.8: `cross_section(span_axis='y', at_a_mm=3.8)`. Example: `cross_section(axis="z", at_mm=10, span_axis="x")` → reads off wall positions along X at Z=10 mm.

`fillet_edges_smart(radius_mm, body_index=0, which="all", min_radius_mm=0.2)`

Fillet that **searches** for a working configuration: for each radius (halving down to `min_radius_mm`) it tries tangent-chain on then off, verifies the volume delta against `~0.1*r^2*L`, rolls back an overshoot, and reports which combination worked. Exists because a blend that fails at every constant radius, every variable radius, and as a chamfer can succeed on the *first* radius once tangent chaining gives it a run-out.

`explain_fillet_error(message)`

Translates raw ACIS/ASM codes (`ASM_BL_END_TOO_CMPLX`, `ASM_ABL_CAP_COMPLEX`, `ASM_ABL_NO_GEOM_EE`, `ASM_BL_UNFIN_SHEET`, `ALL_TOOL_BODY_REFERENCE_LOST`, …) into what actually fixes them.

`list_faces(body_index=0, max_faces=40)`

Faces with a stable **tempId**, centroid (mm), area (mm²), surface type, largest first. The tempId lets you re-select the same face later via `body.findByTempId(tempId)` in raw code — essential for multi-step face work.

`list_edges(body_index=0, max_edges=40)`

Edges with a stable **tempId**, length (mm), midpoint (mm), longest first. Use the tempId to target specific edges for fillets/chamfers in raw code.

Visual feedback

`view_model(orientation="iso", zoom_x_mm=None, zoom_y_mm=None, zoom_z_mm=None, zoom_radius_mm=None)`

Orient (iso/top/**bottom**/front/back/right/left), fit, screenshot, and **return the image**. An unrecognised orientation is an **ERROR**, not a silent fallback to iso. Pass `zoom_*_mm` + `zoom_radius_mm` to frame a small detail — inspecting a 4 mm feature on a 76 mm part is impossible with a whole-model fit.

`set_view(orientation="iso")`

Reposition the camera without returning an image.

Components & occurrences

`create_component(name)`

New empty component as an occurrence at the origin. One per distinct part.

`list_components()`

All occurrences: fullPathName, component name, origin position (mm).

`box_in_component(component_name, length_mm, width_mm, height_mm, ox_mm=0, oy_mm=0, oz_mm=0)`

Box built inside a named component on its own XY plane.

`move_component(component_name, x_mm, y_mm, z_mm)`

Translate an occurrence's origin. Position BEFORE jointing.

`copy_component(component_name, count, dx_mm=0, dy_mm=0, dz_mm=0)`

Place `count` more occurrences of one definition in a linear pattern. Reuses the definition (edit once, all update) — ideal for repeated hardware.

`ground_component(component_name, grounded=True)`

Lock/unlock a component in space. Ground exactly one base per assembly, first.

Joints

Joints reference component **origins** (or explicit points). To pivot at a specific location, design the component so its local origin sits there, or use `joint_at_point`. Position parts first; joints override transforms after.

`rigid_joint(component_a, component_b)`

Lock two components rigidly at their current relative position.

`revolute_joint(component_a, component_b, axis="z")`

One rotational DOF about `axis` (x/y/z). Hinges, lids, arms.

`joint_at_point(component_a, ax_mm, ay_mm, az_mm, component_b, bx_mm, by_mm, bz_mm, motion="rigid", axis="z")`

Joint at explicit local points on each component. `motion`: rigid/revolute.

`list_joints()`

All joints and their motion type.

Export

`export_step(filepath)`

STEP of the whole design. Absolute path, `.step`.

`export_stl(filepath, body_index=0, quality="medium")`

STL of one body for 3D printing. `quality`: low/medium/high.

`export_dxf(filepath, sketch_index=0, component_name="")`

DXF of a sketch profile for laser/waterjet/flat stock. Selects sketch by index in root (or a named component).

Document identity & persistence

> **The single most damaging failure class.** Every tool operates on the **active document**, and Claude cannot see the Fusion window. Three things compound: `doc.dataFile` routes through `app.data` (which can be dead for a whole session), the cheap state is a **cache** that goes stale when the user switches documents in the UI, and `isSaved` alone is ambiguous. Together they produce: overwriting the wrong design, starting a new document instead of continuing in the open one, and reporting a save that never happened.

`document_report()`

Run this before any save, clear, or destructive edit. Authoritative *live* read (not the cache) of the active document and **every other open document**: name, `isSaved` and `isModified` read independently, lineage id, version number, and whether `app.data` was reachable at all.

It ends with an explicit **VERDICT** and — crucially — distinguishes *"never saved"* from *"cannot tell"*. When `app.data` is broken the lineage is unreadable, and treating that as "unsaved" is precisely what causes a duplicate document to be created.

`activate_document(name_or_index)`

Make a specific open document active, by name substring or by the index shown in `document_report`. Use it to **bind to the user's document** rather than assuming the active one is right.

`save_document(name="", create_new=False, force_new_lineage=False)`

Saves and then **verifies**. Never branches on a single flag:

Situation	Action
Lineage readable + present	`doc.save()` — version the **open** document
Lineage readable + absent	`saveAs name` — genuine first save
Lineage **unreadable**	try `doc.save()` **first** (non-destructive; needs no `app.data`). If it fails, **stop** — no silent `saveAs`
`create_new=True`	always `saveAs` — explicit new lineage

Reports version before/after and prints **CONFIRMED** only when an independent signal agrees (version bumped, modified cleared, or a lineage appeared). Otherwise it prints **UNCONFIRMED** and says not to report the document as saved. `force_new_lineage=True` overrides the guard deliberately.

`get_state()`

Cheap cached state, now including **every open document** and its own **snapshot age**. If `fresh=False`, the user may have switched documents since — use `document_report()` before acting.

`list_recent_designs(limit=15)` / `open_document(design_id)` / `list_versions(design_id="")`

All require `app.data`. When it is broken these **cannot be worked around** — ask the user to open or save the design in the Fusion UI rather than retrying.

Typical workflows

Single part: `clear_document` → `set_parameter`(s) → primitive(s) → features (`fillet_edges`, `shell_body`, `tapped_hole`, `sketch_on_face` + `hole_on_face`) → `view_model` → `get_model_state` / `measure_distance` → `export_stl` / `export_dxf`.

Assembly: `create_component` per part → `box_in_component` (or build geometry) → `ground_component` base → `move_component` others → `rigid_joint` / `revolute_joint` / `joint_at_point` → `view_model` → `save_document`.

Resume later: `list_recent_designs` → `open_document(id)` → modify → `save_document` (new version).

Curved-surface features (wrap / emboss / project)

> These exist because every hand-rolled alternative is unreliable on this build: closed splines on curved faces, `createRuledSurface`, loft between non-planar closed splines, and `BRepBodyDefinition` all fail. Use these tools instead.

`wrap_cut(sketch_index=-1, depth_mm=1.0, face_tempids="", protrude=False, component_name="")`

Wrap a flat sketch onto a **non-planar** face and cut to depth (`protrude=True` embosses outward). The primitive for Voronoi / text / texture on curved and revolved bodies. - `sketch_index=-1` uses the most recent sketch. - **Omit** `face_tempids` to auto-target the largest curved face — recommended, because tempIds die the moment an operation splits faces. - Cuts **per-profile with an individual volume check**, reporting `ok / no-op / failed` counts. Read them. - A **no-op profile almost always fell outside the face's angular sector**: one flat sketch only wraps onto roughly a 70–100° sector. Split large patterns into sectors (0/90/180/270°) using `construction_plane_tangent`.

`project_to_surface(sketch_index=-1, face_tempid=0, component_name="")`

Project sketch curves onto a face without cutting — use it to see where a pattern will actually land before committing to `wrap_cut`.

`construction_plane_offset(offset_mm, base="xy", component_name="")`

Offset construction plane from a principal plane. Use this **instead of raw** `ConstructionPlaneInput.setByPlane`, which raises "Environment is not supported" here. Echoes the plane's world origin/normal so a unit slip is visible immediately — `setByOffset` takes **internal cm**, so passing mm places the plane 10× too far and makes subsequent embosses silently no-op.

`construction_plane_tangent(angle_deg=0.0, body_index=0, component_name="")`

Plane tangent to the largest curved face at an angle around its axis. The reliable way to place sketches for **multi-sector wraps** on revolved bodies. Echoes world position/normal.

`sketch_axis_map(sketch_index=-1, component_name="")`

Probe a sketch's local→world mapping via `sketchToModelSpace` and print where local (0,0), (1,0), (0,1) land in world mm, plus a plain-language summary (`local +X -> world -Z`). **Run this before placing geometry on any offset or tangent plane** — local axes rarely match world.

TemporaryBRepManager — the robust path for many cuts

`cut_prisms(prisms, body_index=0, component_name="")`

Cut **many rectangular prisms in ONE timeline feature** using `TemporaryBRepManager`. This is usually a better investment than `wrap_cut` for cell/lattice patterns because it depends on **neither construction planes nor face tempIds** — the two most fragile things in this environment — and nothing is re-selected, so it survives face-splitting.

`prisms` is a JSON list, each entry a centre plus full sizes in mm:

```
[{"x":0,"y":0,"z":5,"w":4,"d":4,"h":20}]
```

Reports volume delta and warns on a no-op. Hundreds of cutters complete as a single operation.

Rolling your own with `tbm()`

Two helpers are injected into every raw-code namespace:

- `tbm()` → `TemporaryBRepManager`. Build throwaway solids in memory, boolean them together, commit one result.

- `commit_tool_body(tool_temp, target_body, keep_tool=False)` → lands a temp solid in the design and cuts it from `target_body` in one combine feature, printing the volume delta and warning on ~zero.

```
t = tbm()
obb = adsk.core.OrientedBoundingBox3D.create(
    adsk.core.Point3D.create(mm(0), mm(0), mm(5)),
    adsk.core.Vector3D.create(1,0,0), adsk.core.Vector3D.create(0,1,0),
    mm(4), mm(4), mm(20))
tool = t.createBox(obb)
# union more cutters into `tool` ...
commit_tool_body(tool, root.bRepBodies.item(0))
```

Why it wins here: no construction planes (`setByPlane` and `ConstructionPoints.add` are environment-blocked), no tempId re-selection, and one timeline feature instead of N.

Edge selection & fillet verification

`which=all/vertical/top` cannot express what real fillets need — *"the edges between surface type A and B, within a z range, longer than L."* These tools do.

`select_edges(body_index=0, face_pair="", z_min_mm=None, z_max_mm=None, min_len_mm=None, max_len_mm=None, component_name="")`

Select edges by predicate; returns a comma-separated **tempId list** ready to paste into `fillet_edges_smart(edge_tempids=...)`. - `face_pair` — substring match on the adjacent surface-type pair, e.g. `"Cylinder|Cone"`, or just `"Cone"` for any edge touching a cone. - `z_min_mm` / `z_max_mm` — edge **midpoint** z range. - `min_len_mm` / `max_len_mm` — length range.

`list_edges` now reports the same **surface-type pair** per edge, so you don't have to re-implement the classifier.

`fillet_preview(radius_mm, edge_tempids, body_index=0)`

What a fillet *would* touch, before committing: selected edges, adjacent faces, and any face smaller than **~4r²** — the signature of a radius too large for the feature it lands on, which is how a fillet silently **consumes** a narrow feature (a 1.3 mm rim annulus, say) while the volume delta still looks fine.

`fillet_edges_smart(radius_mm, body_index=0, which="all", min_radius_mm=0.2, edge_tempids="")`

Searches radius × tangent-chain and verifies **two** ways: - **Volume** — delta vs `~0.1·r²·L`, catching a blend that propagated and *added* geometry far away. - **Topology** — which faces *disappeared*, warning when a face smaller than `4r²` was consumed. This is the opposite failure and volume checks miss it entirely.

Overshoots are rolled back and the search continues. **A zero delta at every radius is reported as a RESULT, not a failure** — it means the surfaces already meet tangentially and need no fillet, which is real design information.

`force_delete_feature(feature_name)`

Escape hatch for a **mutual-reference deadlock** — two features each reporting the other as the blocking failure, where neither suppress nor `deleteMe()` works in either order. Does what the Fusion UI does: rolls the timeline marker to just *before* the feature, deletes it there, rolls forward, and reports what else became unhealthy.

Sweeps and splines

`sweep_tube_check(path_sketch_index=-1, tube_radius_mm=1.0, samples=60, component_name="")`

Check a sweep **path** before sweeping. `ASM_SELF_INTER` on a circular-profile sweep blames the profile or taper angle, but the cause is almost always the path: if its minimum curvature radius is below the tube radius, the swept body self-intersects. Reports minimum curvature radius, where it occurs, and total path length.

`respline(points_json, sketch_index=-1, curve_index=0, plane="xy", component_name="")`

Rebuild a fitted spline with a **new** set of fit points — Fusion lets you move existing fit points but not add one. Points are given in **world mm** and mapped onto the chosen plane using the measured mappings. **Limitation, stated honestly:** recreating a curve *breaks* downstream references; Fusion has no true in-place respline. The tool reports every feature that became unhealthy so you can see the repair surface.

Batching

`run_batch(operations_json, stop_on_error=True)` fuses several operations into ONE round trip and one Fusion job. Only **geometry and parameter** tools are batchable (an explicit allowlist). Reporting, async and document-level tools — `clear_document`, `save_document`, `view_model`,

`document_report`, … — must be called directly so their results and safety checks are not swallowed. An invalid batch is rejected before anything executes.

Verification guarantees (what "success" means)

The most expensive failures in this project were **silent wrong-target** and **unverified success** — a tool reporting that it did something it did not do. These are now checked rather than assumed:

Operation	What is verified
Any geometry tool	The active product **is a Design**. In a Drawing/CAM/Render workspace you get a clear error naming the workspace and document, not an obscure `.rootComponent` failure.
`save_document`	Version bump / modified-flag cleared / lineage appeared → **CONFIRMED**, else **UNCONFIRMED**.
`export_step` / `export_stl` / `export_dxf`	`execute()` return **and** the file exists with non-zero size → **CONFIRMED**, else an explicit failure.
`set_parameter`	The stored expression is **read back** and compared. Fusion silently rejects or clamps expressions it dislikes.
`rigid_joint` / `revolute_joint` / `joint_at_point`	Joint **count increased**.
`drill_hole` / `hole_on_face` / `tapped_hole` / `shell_body`	Volume delta, with a no-op warning.
`fillet_edges` / `fillet_edges_smart`	Volume delta **and** face topology (consumed faces).
`cut_prisms`	Volume delta.
`clear_document`	States the target **document name and lineage before destroying anything**.

Rule of thumb: if a tool prints UNCONFIRMED or a `***` warning, do not report the action as done. Those messages exist because the underlying API call returned successfully while achieving nothing.

API traps on this build (measured, not guessed)

Each of these cost real debugging time. They fail **quietly** — valid bodies, success messages, plausible bounding boxes — so they're worth reading once.

Sketch-plane mapping: the YZ plane is not what you'd expect

Measured with `sketchToModelSpace`:

Plane	Mapping	Helper
XY	`sketch(u,v) -> world(u, v, 0)`	`P_xy(x,y)`
XZ	`sketch(u,v) -> world(u, 0, -v)`	`P_xz(x,z)`
YZ	`sketch(u,v) -> world(0, v, -u)`	`P_yz(y,z)`

The YZ row is **sketch X → world −Z, sketch Y → world +Y** — *not* the "sketch X → world Y, sketch Y → world Z" that earlier docs claimed. A wrong mapping produces a body with a correct bounding box and a correct profile area that only reveals itself several operations later.

Don't reason about it — use `P_xy` / `P_xz` / `P_yz` (they take world mm and return the right sketch point), and call `verify_plane_mapping(sketch)` once on any new, offset, or tangent plane to print its measured basis.

Degenerate edges crash edge iteration

Two lofts meeting tangentially can leave a **zero-length edge with fewer than two faces**; `e.faces.item(1)` then raises `Bad index parameter`. Guard every walk of `body.edges`:

```
if e.faces.count < 2 or to_mm(e.length) < 1e-4: continue
```

`edge_ok(e)` and `safe_edges(body)` are injected, and `list_edges` / `select_edges` / `fillet_edges_smart` all apply the guard internally.

`computeAll()` does not guarantee a fresh `body.volume`

After editing sketch points, `body.volume` and `boundingBox` can read **byte-identical** to before, even after `computeAll()`. An identical volume does **not** prove the edit failed. Use the injected `refresh_geometry()` (recomputes and reports whether volumes moved), and confirm with `cross_section` or point sampling.

`app.data` can be dead for an entire session

`InternalValidationError : group` on any `app.data` access survives a Fusion restart and kills `list_recent_designs`, `open_document`, and `saveAs`. `save_document` detects it and falls back to `doc.save()` on an already-saved document. **`open_document` and `list_recent_designs` cannot be worked around** — they need `app.data`. Have the user open the design in the Fusion UI.

`TimelineObject` has no `deleteMe`

Use `tl.item(i).entity.deleteMe()`. The TimelineObject exposes only `entity, rollTo, reorder, isSuppressed, healthState`.

Bodies added via `BaseFeature` return a STALE reference

```
bf = root.features.baseFeatures.add()
bf.startEdit()
body = root.bRepBodies.add(tempBody, bf)
name = body.name
bf.finishEdit()
# `body` is now unusable -> RuntimeError: ALL_TOOL_BODY_REFERENCE_LOST
body = next(b for b in root.bRepBodies if b.name == name)   # re-fetch
```

Renaming through the stale handle also silently doesn't stick. `cut_prisms` and `commit_tool_body()` handle this internally.

Sketch entity properties return fresh wrappers — `is` is a trap

```
arc.startSketchPoint is arc.startSketchPoint   # False!
```

Using `is` to pick which end of an arc to connect to silently selects the same point twice, giving an ambiguous chain and a sweep that produces a fraction of the expected volume with no error. Compare `.geometry` coordinates instead, and **print** `path.count` after `Path.create` — a mis-chain is invisible otherwise.

`addByCenterStartSweep` normalizes start/end

With a negative sweep angle, `startSketchPoint` comes back as the geometric *end* of what you asked for. Prefer `addByThreePoints` with explicitly computed points.

`setMidRadii` takes Python lists, not `ObjectCollection`

```
vs.setMidRadii([ValueInput.createByReal(mm(0.5))],
               [ValueInput.createByReal(0.5)])
```

Passing ObjectCollections gives a raw SWIG error with no hint.

`app.data` can fail for an entire session

`InternalValidationError : group` on `app.data.activeProject` disables `list_recent_designs`, `open_document`, and any `saveAs`. It can persist across a Fusion restart. `save_document` now detects this and falls back to `doc.save()` — which works on an already-saved document — and tells you a new lineage can't be created. Create the new document in the Fusion UI first.

Fillets can destroy geometry silently

A tangent-chained fillet can propagate down a tangent-continuous surface and fill in a relief tens of mm away, reverting a bore radius, while reporting success with a valid body and no slivers. `fillet_edges` now checks the volume delta against `~0.1*r^2*L` and warns on an order-of-magnitude overshoot; the same check is available as `fillet_volume_sanity()` in raw code. `isTangentChain` is genuinely double-edged — it also *rescues* blends that have no run-out — so **verify, don't just try**: that's what `fillet_edges_smart` does.

Replacing a body (recommended over clearing)

Don't reach for `clear_document` to start over. The safer idiom keeps history and can't destroy unrelated work:

```
# build the new body first, then retire the old one
retire_body(old_body)      # -> root.features.removeFeatures.add(old_body)
```

`retire_body()` is injected into every raw-code namespace. `clear_document` is still available (now reverse-order and async), but replacement is strictly safer.

Known-unstable operations (refused by the listener)

The listener actively **refuses** these calls before they reach Fusion, because they have hard-crashed the host in practice (which kills the listener and requires a manual add-in restart). If you send code containing them, you'll get a `REFUSED (stability guard)` message instead of a crash:

Call	Why refused	Use instead
`*.defineAsFreeMove(...)` (move-face free move)	Hard-crashes the Fusion host	Edit the parametric feature, or rebuild the body
`*.defineAsOffset(...)`	Not reliably available on this build	`offsetFeatures`, or a shell / offset-plane extrude

If you find another operation that reliably crashes the host, add its call signature to `_DANGEROUS_PATTERNS` in `MCPListener.py` and document it here.

Diagnosing a hang or suspected crash

The listener runs jobs **asynchronously** and answers ping/state on a separate channel, so you can always tell what's happening:

1. `heartbeat()` (or `ping_fusion()`). If it responds with `busy=True`, a job is still running — `job_status(current_job)` for its partial output. If it errors/times out, the listener is down.
2. If a `run_fusion_code` call returned "STILL RUNNING" or a timeout envelope, the job kept going — poll `job_status(job_id)` rather than re-issuing it (re-issuing stacks another job behind the stuck one).
3. `get_state()` is always cheap and never blocks — use it to see the active document and counts even mid-operation.
4. If the listener is truly down: in Fusion, **Shift+S → Add-Ins → MCPListener → Run** to restart it, then `get_state()` to see what survived.

A job that exceeds its budget is flagged `timeout` in the envelope (the watchdog can't force-kill a hung main thread, but you get a clear signal instead of a dead channel).